

Curso de Especialización

Ciberseguridad en las TIC

Ejercicio-DEVSECOPS-ONLINE

Objetivos

- Analizar por fases DevSecOps dónde hay incumplimientos.
- Identificar en qué fase o fases se incumple la seguridad.
- Descubrir qué riesgo concreto existe.
- Saber cómo debería corregirse en cada fase.

Resumen

Autora: Marina del Pilar López Oliva

Marzo 2026

IES Camp de Morvedre

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Revisión

Revisión	Fecha	Descripción
1.0	04/03/2026	Realización del ejercicio.

Índice de Contenidos

Analiza por fases DevSecOps dónde hay incumplimientos.....	4
Plan.....	4
Code.....	4
Build.....	5
Test.....	5
Deploy.....	5
Monitor.....	5

Analiza por fases DevSecOps dónde hay incumplimientos

He consultado los siguientes enlaces para la realización de la práctica:

<https://security.snyk.io/package/pip/flask/1.0.1>

<https://security.snyk.io/package/pip/flask-cors/3.0.8>

Plan

No se muestra que se definieron requisitos de seguridad. No parece que se hayan analizado las amenazas antes de realizar el código, puesto que se han usado versiones inseguras de flask y flask.cors. El login presente es vulnerable, lo que demuestra que no hubo requisitos de autenticación segura. No tiene una validación de entradas, parece que no ha sido planificado. No parece que tenga política de contraseñas clara. Al manejar credenciales, tendría que haber requisitos RGPD/LOPD.

Qué cambiaría:

- Definir requisitos de seguridad medibles, por ejemplo: "las contraseñas deben almacenarse hasheadas".
- Realizar un análisis de amenazas documentado antes de realizar el código.
- Incluir requisitos de cumplimiento normativo para el manejo de credenciales.

Code

El código presenta un SQL Injection puesto que pide contraseña y usuario pero no se escapan los caracteres para que el usuario no ponga un script SQL para borrar, por ejemplo, todo la base de datos. Además, no hay autenticación ni autorización, tampoco existe una gestión de sesiones. Tras el login correcto, no se crea ninguna sesión.. Las contraseñas viajan en modo plano en la consulta de login y se almacenan en la base de datos en texto plano. Tampoco se utiliza ningún tipo de validación. Se concatenan los datos con una base de datos llamada "users.db". El "debug=true" está en el código fuente, lo que puede permitir ejecución remota de código.

Que cambiaría:

- Usar consultas parametrizadas con el parámetro "?".
- Hashear las contraseñas con alguna librería.
- Implementar gestión de sesiones.
- Añadir rate limiting para evitar ataques de fuerza bruta.

- Eliminar el debug=true del código o ponerlo en false.

Build

Se hicieron pruebas con una herramienta SCA para esta fase. Se detectaron versiones vulnerables: flask==1.0 y flask-cors==3.0.8. Se nos indica que existen versiones más recientes con parches de seguridad para flask. Para flask-cors se nos indica que se debería actualizar a 3.0.9 o superior. También se nos indica que el CORS está sin restricciones y cualquier origen puede hacer peticiones al backend, lo que facilita la explotación. No se evidencia el fichero requirements.txt que se menciona en el enunciado, pero no se visualiza. El requirements.txt debería incluir versiones fijadas con hashes, lo que hace que el build no sea susceptible a ataques de cadena de suministro. Aparte del SCA debería realizar un análisis SAST que hubiera detectado el SQL Injection.

Qué cambiaría:

- Integrar SCA y SAST como paso obligatorio que detectes vulnerabilidades críticas sobre todo.
- Actualizar las versiones de flask y flask-cors.
- Restringir CORS a el dominio que queramos.
- Ejecutar un checklist automático.

Test

No se muestra que se hayan probado inputs maliciosos ni que se haya interceptado el tráfico antes de la producción con herramientas como Burp. No se probó el bypass de login ni se probó la manipulación de parámetros. Tampoco se mencionan informes de pentesting realizados antes de la producción. No hay test unitarios ni de integración básicos. En los logs se observa un SQL Injection que se ha realizado ya en producción, lo que es una consecuencia de no haber hecho test de seguridad antes.

Qué cambiaría:

- Ejecutar un análisis DAST antes del release.
- Probar específicamente las vulnerabilidades encontradas anteriormente y algunas básicas como SQL Injection, bypass de login y CORS abuse.
- Establecer criterios de aceptación de seguridad para no pasar a release sin realizar las pruebas o teniendo vulnerabilidades críticas sin solucionar.

Release

Se ha publicado la aplicación sin una revisión final. Se ha saltado la aprobación, porque en las pruebas realizadas ya habíamos visto vulnerabilidades, pero aún así la desplegaron. En DevSecOps existe un security gate que debería haber bloqueado el release automáticamente al tener CVEs críticos sin resolver del SCA. No se realizaron tests antes de lanzarlas para comprobar la seguridad de la aplicación. No se cambiaron las versiones y por lo tanto tiene vulnerabilidades conocidas. Tampoco se validó un checklist de seguridad.

Qué cambiaría:

- Implementar un security gate automático para que si el SCA o SAST detectan vulnerabilidades críticas, el release se bloquee.
- Exigir aprobación explícita del responsable de seguridad antes del despliegue.
- Gestionar las versiones y hacer un checklist de seguridad para ver que se han corregido correctamente las versiones.

Deploy

Se desplegó la aplicación con el modo debug activado y todos los puertos abiertos disponibles para todos los lugares. Utiliza HTTP en lugar de HTTPS. Tiene las credenciales en texto plano y guardadas directamente en la base de datos. No estaba el código seguro pero la configuración tampoco. Y se expuso la base de datos users.db sin ninguna protección. El servidor de desarrollo de flask no debe usarse en producción. No hay variables de entorno para configuración sensible.

Que cambiaría:

- Usar un servidor de desarrollo que no sea el de flask en producción.
- Gestionar secretos con variables de entorno.
- Añadir HTTPS.
- Restringir el acceso porque el "0.0.0.0" expone la app a toda la red.

Monitor

Los logs registran la actividad pero también exponen la query de SQL completa con los datos del ataque en texto plano. Se detecta un ataque de SQL injection. Se descubre la brecha el mismo día que se lanza. No hay un SIEM configurado ni alertas automáticas. No hay un plan de respuesta a incidentes.

Que cambiaría:

- Configurar alertas automáticas.
- No poner en los logs datos sensibles.
- Definir un plan de respuesta a incidentes.
- Integrar un SIEM.